M. Yasodha Krishna
192472100

SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: MLA03
COURSE CODE: Reinforcement learning for Environment and Agent

---

COURSE OUTCOME COVERED

CO1: *Analyze reinforcement learning frameworks and Markov Decision Process concepts to model decision-making problems.(BL2, BL3)*

Assessment Tool Weightage: 30%

Dr Dumpala Prasanth

QUESTIONS: 10 **Total Marks:50**

Annexure A

Constraint - Based Problem-Solving **Duration: 2Hrs**

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

(10 Marks)

2. Implement an ε-greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ε values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

(10 Marks)

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

(10 Marks)

6. Implement a Reinforcement Learning agent that controls traffic signals at an intersection while ensuring that emergency vehicles receive immediate priority. Explain how exploration, exploitation, and policy learning are modified to satisfy the safety constraint.

(10 Marks)

7. Develop a Python program using Keras/TensorFlow to simulate a reinforcement learning agent for packet routing in a wireless network with limited bandwidth. Explain how bandwidth constraints influence the reward function and policy optimization.

(10 Marks)

8. Analyse an experimental comparison between random action selection and ε-greedy action selection in a Multi-Armed Bandit problem under a fixed time constraint. Record cumulative rewards and explain which strategy provides better performance.

(10 Marks)

9. Design a Reinforcement Learning framework for a delivery drone that must maximize successful deliveries while avoiding no-fly zones and maintaining battery limits. Explain the MDP formulation, Bellman equation, and policy learning process.

(10 Marks)

10. Implement a Reinforcement Learning solution using Python for a smart energy management system where electricity consumption must remain below a specified threshold. Explain how reward shaping, policy learning, and feasibility constraints improve decision-making.

(10 Marks)

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts
Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

M. Yasodha Krishna
192422100
SEM-A FILA0303

CO1 - AT3

ASSESSMENT

1) Grid world navigation problem

State space

- Agent position in the grid

Action space

- UP, down, left right

Reward function

- Goal reached: +100
- Hit obstacle: -50
- Normal move: -1

Constraints

- Agent cannot move outside the grid
- Agent must avoid obstacles
- Agent should reach the goal in minimum steps

Python code

```
import numpy as np
goal = (4,4)
state = (0,0)
while state != goal:
    state = (state[0]+1, state[1])
    Print("Current state:", state)
Print("Goal reached")
```

Conclusion

Q-learning helps constraints to learn short and safest path.

ε - Greedy Multi - Armed Bandit

objective

Maximize rewards within a budget of 500 trails

ε value comparision

ε	Exploration	Exploitation
0.1	low	High
0.3	medium	medium
0.5	High	low

constraint

only 500 trails are available

Python code

```
import random
epsilon = 0.1
if random.random() < epsilon:
    print("Explore")
else:
    print("Exploit")
```

conclusion

ε = 0.1 usually gives higher rewards because it exploits the best arm more frequently

3. ### MDP for Autonomous Robot

states

- start
- intermediate locations
- Goal

Actions

- Move Forward, Turn left, Turn right

Rewards

- Reach goal = +100
- movement cost = -2
- High energy usage = -10

Energy constraint

Battery energy must be not exceed the available limit

Python code

```
states = ["start", "path", "Goal"]
rewards = {"start": 0, "path": -2, "Goal": 100}
for s in states:
    print(s, rewards[s])
```

conclusion

The robot learns the path that minimizes energy consumption and reaches the destination efficiently

4. value iteration in constrained grid

Restricted states

certain grid cells cannot be entered

Bellman Equation

$V(s) = \max_a [R + \gamma V(s')]$

constraints

- Restricted states must be avoided
- Goal state must be reached

Python code

```
gamma = 0.9
reward = 10
value = 0
for i in range(5):
    value = reward + gamma * value
print(value)
```

Conclusion

Value iteration repeatedly updates state value until the optimal policy is found.

5. Warehouse Robot

constraints

- Battery capacity = 30 minutes
- Maximum delivery must be completed before battery dies

Reward Design

- successfully delivery = +20
- Battery saving = +5
- Battery exhausted

Python code

```
battery = 30
if battery > 5:
else: print ("delivery")
    print ("return")
```

Conclusion

The RL agent learns to maximize delivery while staying within batteries.

6. Traffic signal control

States

- Traffic density, signal status, Emergency vehicle Presence

Actions

- Green north-south, Green East-West, Emergency override

Safety constraint

Emergency vehicles must get immediate priority

Reward

- Reduced waiting time = Positive reward
- Emergency delay = large penalty

Conclusion

The learned policy reduces congestion while ensuring emergency vehicle safety.

7. Packet Routing in wireless network

Constraints

limited bandwidth availability

States

- Network load
- Available bandwidth

Actions

- Select routing path

Reward Function

Reward = Throughput − Delay − Packet loss

conclusion

The RL agent chooses routes that maximize throughput and minimize congestion

8. Random vs ε- Greedy selection

Random strategy

- Selects actions randomly
- No learning occurs

ε- Greedy strategy

- Balances exploration and exploitation.
- Learns the best action over time

Result

ε- Greedy is more cumulate rewards than random selection

conclusion

ε- Greedy is more effective under fixed time constraints

9. Delivery drone framework

states

- location
- Battery level
- No- Fly zone status

Actions

- Move in different directions
- Deliver package

Constraints

- Avoid no-fly zones.
- Maintain sufficient battery.

Reward

- Successful delivery = +100
- Enter no-fly zone = -100
- Battery exhausted = -50

Conclusion

The drone learns safe and efficient delivery routes

10. Smart Energy management system

Objective

Keep electricity consumption below a threshold

States

- Energy demand
- Appliance state
- Electricity price

Actions

- Turn ON appliance
- Turn OFF appliance
- Reduce load

Reward Design

- Energy saved = Positive reward
- Exceed threshold = Negative reward

constraints

Electricity consumption must remain bellow the specified limit

conclusion

Reward shaping helps the agent learn energy-effici-ent decision while satisfying consumption constraint